

# Inductive Process Discovery from Partially Ordered Event Data

HUMAM KOURANI<sup>✉</sup>, Fraunhofer Institute for Applied Information Technology FIT, Germany

TOM BREUER, RWTH Aachen University, Germany

GYUNAM PARK, Fraunhofer Institute for Applied Information Technology FIT, Germany

WIL M. P. VAN DER AALST, RWTH Aachen University, Germany

The Inductive Miner (IM) family is a prominent class of process discovery techniques, combining efficient recursive decomposition with soundness-by-construction guarantees. However, IM techniques usually assume traces to be totally ordered sequences of activity occurrences. This assumption is convenient, but can introduce systematic bias: activities may have durations, events may share coarse timestamps, or the data may constrain only some event pairs. Forcing such executions into arbitrary sequences hides inherent concurrency and may introduce sequential dependencies that were never observed as causal constraints. Partial orders provide a more faithful representation, but integrating them into IM discovery is challenging because standard abstractions are sequence-based; directly reusing them would require linearizing each partial order, which becomes prohibitively expensive under high concurrency. We introduce a lifting of IM discovery from total orders to partially ordered traces. Instead of redesigning the miner and its cut detection logic, we redefine the trace abstraction layer and the recursive projections to operate directly on partial orders. The approach is conservative over totally ordered traces, avoids linearization explosion, and preserves the recursive structure and guarantees that make IM attractive. Experimental results show that the proposed lifting avoids the combinatorial overhead of linearization, reduces sensitivity to arbitrary tie-breaking in timestamped event data, and allows process behavior to be learned from fewer observations by preserving concurrency at the trace level.

Additional Key Words and Phrases: process discovery, Inductive Miner, partially ordered traces, POWL, concurrency

## 1 Introduction

Process discovery is one of the central tasks of process mining. Given event data recorded by information systems, the goal is to derive a process model that describes the observed behavior and can subsequently be used for communication, conformance checking, simulation, automation, and improvement initiatives [26]. Among the many discovery paradigms proposed over the years, the Inductive Miner has become a reference point because it combines scalability with formal guarantees. It recursively decomposes the event log using cuts over directly-follows abstractions and constructs a hierarchical model that is sound by construction [17]. This idea has also been lifted to more expressive intermediate notations, such as the Partially Ordered Workflow Language (POWL), where partial orders and choice graphs can capture non-block-structured concurrency and decisions while retaining the inductive discovery scheme [11, 14, 15].

Despite these advances, the input side of most inductive discovery algorithms remains predominantly sequential. An event log is typically represented as a multiset of traces, and each trace is a sequence of activity labels. This representation is simple and efficient, but it silently assumes that all relevant events in a case can be placed in one reliable total order. This assumption is often questionable. Activities may overlap in time. Start and complete lifecycle transitions may show that one task was still running while another was started [18]. Several events may share the same timestamp because of coarse temporal granularity. During event abstraction, analysts may intentionally discard

---

Authors' Contact Information: Humam Kourani, [humam.kourani@fit.fraunhofer.de](mailto:humam.kourani@fit.fraunhofer.de), Fraunhofer Institute for Applied Information Technology FIT, Sankt Augustin, Germany; Tom Breuer, [tom.breuer@rwth-aachen.de](mailto:tom.breuer@rwth-aachen.de), RWTH Aachen University, Aachen, Germany; Gyunam Park, [gyunam.park@fit.fraunhofer.de](mailto:gyunam.park@fit.fraunhofer.de), Fraunhofer Institute for Applied Information Technology FIT, Sankt Augustin, Germany; Wil M. P. van der Aalst, [wvdaalst@pads.rwth-aachen.de](mailto:wvdaalst@pads.rwth-aachen.de), RWTH Aachen University, Aachen, Germany.

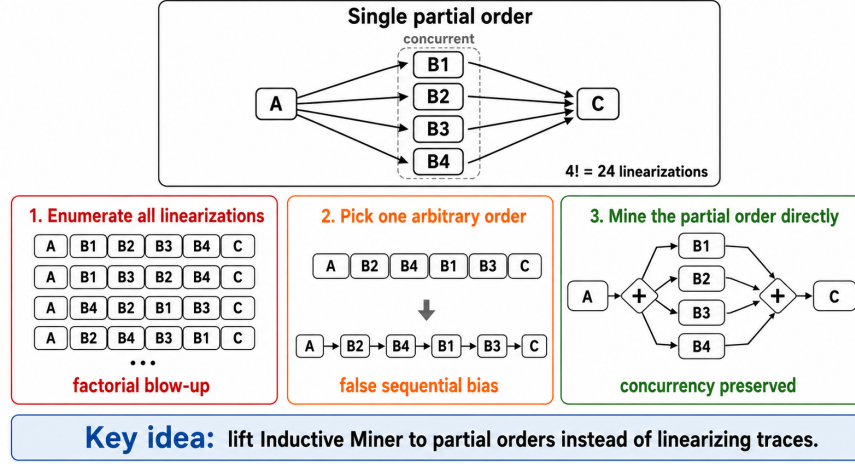


Fig. 1. Overview of discovery from partially ordered traces. The top panel shows a single partially ordered trace in which activity  $A$  is followed by four concurrent activities  $B_1, B_2, B_3, B_4$ , which are all followed by  $C$ . This single partial order compactly represents  $4! = 24$  possible linearizations. Enumerating all linearizations leads to factorial growth, while selecting one arbitrary linearization may introduce false sequential bias. Our approach lifts Inductive Miner to operate directly on partially ordered traces, preserving concurrency in the discovered model.

accidental ordering evidence and keep only robust precedence constraints. In all these situations, a partially ordered trace is the more faithful representation: some event pairs are ordered, while other pairs are intentionally left incomparable.

The practical impact of forced total ordering is not merely notational. If two activities are concurrent but the log stores one arbitrary sequence per case, discovery sees an artificial order. When the arbitrary order is consistent, the discovered model may impose a false sequential dependency. This problem has motivated a growing body of work on partially ordered event data, partial-order variants, and partial-order-based process mining [19]. However, there is still a gap between the availability of partial-order event data and the reuse of mature, scalable inductive discovery machinery.

A straightforward approach is to enumerate all linearizations of each partially ordered trace and feed the resulting sequential log into an existing miner. This solution is conceptually attractive because it requires no change to the miner, but it comes with substantial computational overhead. A partial order with a concurrency of size  $k$  already represents  $k!$  permutations, so the number of generated traces can grow factorially with the degree of concurrency. Enumerating these permutations can also change the statistical meaning of the log: one observed execution with many concurrent events may dominate the frequency counts merely because it has many linear extensions. Sampling linearizations reduces the overhead, but introduces variance and may miss orderings that are important for detecting concurrency.

Figure 1 illustrates the core challenge and the proposed solution: instead of linearizing partially ordered traces, we lift the abstraction layer of the Inductive Miner to partial orders. The central design choice is simple: cuts should still be detected over directly-follows and eventually-follows style abstractions, but these abstractions should be computed from partial orders rather than sequences. Since IM is recursive, we also lift projection: after a cut is detected, each sublog is obtained by taking the induced sub-order on the events of the corresponding activity block. Thus, partial-order information is preserved throughout the recursion, not only in the initial abstraction.

We provide an implementation of the proposed approach in the Python library POWL (<https://pypi.org/project/powl/>) and evaluate it from three complementary perspectives. The experiments show that direct discovery from partially ordered traces preserves concurrency evidence without the combinatorial overhead of enumerating linearizations, reduces sensitivity to arbitrary ordering choices in timestamped event data, and can capture the behavior of a concurrent process from fewer observed executions than discovery from sequential traces.

The remainder of the paper is structured as follows. [Section 2](#) reviews related work, while [Section 3](#) introduces the required concepts and notation. [Section 4](#) presents the proposed discovery approach, including the lifted abstraction and projection mechanisms. [Section 5](#) evaluates the approach. Finally, [Section 6](#) summarizes the findings and outlines directions for future work.

## 2 Related Work

*Inductive process discovery.* The Inductive Miner discovers block-structured models by recursively detecting cuts over log abstractions [17]. Its appeal lies in soundness by construction, scalability, and a clear decomposition principle. Several extensions improve robustness, frequency handling, and expressiveness. POWL discovery extends the intermediate representation from process trees to partial orders [13, 15], and choice graphs further relax block-structured decision constraints [12].

*Concurrency and partial-order event data.* The treatment of inherent concurrency in event data is a growing topic in process mining [19]. Some techniques exploit lifecycle or interval information to distinguish concurrency from sequential interleavings, including lifecycle-aware Inductive Miner variants [18] and extensions of Split Miner for overlapping activity intervals [1]. In [21], partial orders are used to evaluate student behavior, where course-taking histories are modeled without imposing unnecessary sequential dependencies. Other work uses richer behavioral representations such as prime event structures [2, 7]. Our proposed approach is orthogonal to these approaches: it assumes that concurrency has already been represented as partially ordered traces and focuses on consuming such traces within the IM framework.

*Discovery from partial orders.* Several approaches discover models directly from partially ordered behavior, including aggregation of instance graphs or causal runs [6, 25], ILP-based discovery [8], replay- and synthesis-based mining [9], region-based synthesis [16], unfolding-based synthesis using independence relations [20], and folding of partially ordered runs [3]. These methods demonstrate the breadth of partial-order-based discovery, but they generally follow synthesis or aggregation paradigms rather than recursive IM-style decomposition. Such methods can also face scalability challenges on large or complex logs [19].

*Partial-order variants in Cortado.* Partial-order tools such as Cortado support partially ordered event data for variant representation, visualization, querying, and discovery [22–24]. In Cortado, concurrent variants are sequentialized before applying process discovery, whereas our proposed approach lifts the abstraction and projection steps of IM so that discovery operates directly on partially ordered traces.

*Inductive Miner - Lifecycle.* The closest related work within the Inductive Miner family is the lifecycle-aware Inductive Miner (IMLC) [18]. IMLC assumes lifecycle-enriched event logs in which every activity execution is represented by start and completion events. For example, the sequential trace  $\langle a_{\text{start}}, b_{\text{start}}, a_{\text{complete}}, b_{\text{complete}} \rangle$  reveals that  $a$  and  $b$  overlap, since  $b$  starts before  $a$  completes. IMLC exploits such overlaps to construct concurrency-enriched abstractions for cut detection, while the underlying event trace remains totally ordered and standard sequence-based IM splitting can be

retained. Our approach removes this total-order assumption at the representation level: the input consists directly of partially ordered activity occurrences, so two events can be incomparable without being encoded through a particular ordering of lifecycle markers. Lifecycle intervals are therefore only one possible source of such partial orders; they may also originate from coarse timestamps, domain-specific precedence constraints, or other abstraction mechanisms. Since partially ordered traces have no canonical left-to-right order, we additionally lift recursive projection itself so that partial-order information is preserved throughout discovery.

*Aggregation-based discovery with POWL.* Another related line of work is the partial-order aggregation-based discovery approach in [10]. This approach also uses partially ordered traces as input and produces sound POWL-based models, but it does not use the Inductive Miner framework; it aggregates partial-order behavior directly. This approach may produce more complex models as it allows for duplicating activity labels. The two approaches therefore mainly share their use of POWL as an intermediate representation and their soundness-by-construction guarantees, while relying on different discovery frameworks.

### 3 Preliminaries

#### 3.1 Sequences and Event Logs

Let  $\mathcal{A}$  be a finite universe of activity labels. A *sequence trace* over  $\mathcal{A}$  is a finite sequence  $\sigma = \langle a_1, \dots, a_n \rangle$  with  $a_i \in \mathcal{A}$  for all  $1 \leq i \leq n$ . A traditional event log  $L$  can be abstracted as a multiset of sequence traces. We write  $L(\sigma)$  for the multiplicity of sequence  $\sigma \in L$ .

Given a sequence  $\sigma = \langle a_1, \dots, a_n \rangle$ , the start and end activities are  $a_1$  and  $a_n$  if  $n > 0$ . The directly-follows support of  $\sigma$  is

$$\text{DFG}(\sigma) = \{(a_i, a_{i+1}) \mid 1 \leq i < n\}.$$

The eventually-follows support is

$$\text{EFG}(\sigma) = \{(a_i, a_j) \mid 1 \leq i < j \leq n\}.$$

In practical discovery algorithms these supports are lifted to frequency counters over a multiset of traces. The Inductive Miner uses such abstractions to detect cuts corresponding to control-flow structures [11, 17].

#### 3.2 Partially Ordered Traces

The input considered in this paper generalizes sequence traces. Instead of assuming that all events of a case are totally ordered, we allow partial orders.

*Definition 3.1 (Partially ordered trace (POT)).* A *partially ordered trace* over  $\mathcal{A}$  is a tuple

$$\pi = (E, \prec, \lambda),$$

where  $E$  is a finite set of events,  $\prec \subseteq E \times E$  is an irreflexive and transitive relation (i.e., a partial order), and  $\lambda : E \rightarrow \mathcal{A}$  maps events to activity labels. If neither  $e \prec f$  nor  $f \prec e$  holds, then  $e$  and  $f$  are *incomparable*, written  $e \parallel f$ . A *partially ordered event log (POT log)* is a multiset of partially ordered traces.

The relation  $\prec$  captures forced precedence constraints. Incomparable events may represent intentionally preserved concurrency or uncertainty. For a POT  $\pi = (E, \prec, \lambda)$ , the *transitive reduction* (also called *cover relation*) is defined as

$$\prec^- = \{(e, f) \mid e \prec f \wedge \nexists g \in E : e \prec g \wedge g \prec f\}.$$

The minimal and maximal events are

$$\min(\pi) = \{e \in E \mid \nexists f \in E : f \prec e\}, \quad \max(\pi) = \{e \in E \mid \nexists f \in E : e \prec f\}.$$

A *topological ordering* of a finite partial order  $(E, \prec)$  is a sequence  $\langle e_1, \dots, e_n \rangle$  containing each event in  $E$  exactly once such that, for all  $e_i, e_j \in E$ , if  $e_i \prec e_j$ , then  $i < j$ .

*Definition 3.2 (Linearization).* Let  $\pi = (E, \prec, \lambda)$  be a partially ordered trace. A sequence  $\sigma = \langle \lambda(e_1), \dots, \lambda(e_n) \rangle$  is a *linearization* of  $\pi$  if  $\langle e_1, \dots, e_n \rangle$  is a topological ordering of  $(E, \prec)$ . The set of all linearizations of  $\pi$  is denoted by  $\text{Lin}(\pi)$ .

A partial order compactly represents a possibly large set of total orders. If  $k$  events are pairwise incomparable, the partial order has  $k!$  linearizations. This is the source of the linearization explosion that our proposed approach avoids.

### 3.3 POWL and Inductive Discovery

POWL is a hierarchical modeling language that combines the advantages of process trees with partial orders [14, 15]. Recent POWL variants also support choice graphs to represent non-block-structured decision behavior [11, 12]. The important aspect for this paper is that POWL discovery already follows the inductive decomposition paradigm: a log abstraction is computed, a cut is detected, the log is projected onto the cut blocks, and the algorithm recursively discovers submodels.

This recursive scheme can be summarized as follows. Given an input object  $X$ , first test base cases such as an empty log or a single activity. If no base case applies, compute a cut. A cut is a partition or ordered grouping of the activity alphabet together with an operator specification. For example, a sequence cut with groups  $A_1, \dots, A_k$  yields a sequential composition of recursively mined projections. If no cut is found, a fall-through model is returned. The strength of this scheme is that the discovered model is composed recursively and can retain soundness guarantees when the modeling constructs are sound by construction.

The classical version assumes that the input object is a sequence log or a directly-follows graph derived from a sequence log. Our proposal changes the derivation of this input object when traces are partial orders. Note that the abstraction proposed in this paper is not limited to the POWL Inductive Miner: any variant of the inductive miner whose cut detectors consume directly-follows and eventually-follows information can adopt the same lifting principle.

## 4 POT-Aware Inductive Mining

This section presents the lifting of inductive discovery from totally ordered traces to partially ordered traces. Section 4.1 introduces the POT abstraction artifacts used by the miner. Section 4.2 then defines the projection operations needed for recursive mining. Finally, Section 4.3 summarizes the complete pipeline, including the extraction of partially ordered traces from timestamped event data and the recursive discovery procedure that reuses existing inductive cut detection mechanisms.

### 4.1 POT Abstraction Artifacts

For sequence logs, inductive miners derive their control-flow evidence from log-level artifacts such as start activities, end activities, directly-follows relations, and eventually-follows relations. We lift these artifacts to POT logs by interpreting the partial order of each trace directly, rather than first choosing or enumerating compatible total orders. The resulting artifacts expose the precedence constraints and incomparabilities that are relevant for cut detection.

**4.1.1 Start and end counters.** For a sequence trace, there is at most one start and one end activity. For a partially ordered trace there may be several minimal and maximal events. We therefore define start and end evidence as sets, lifted to counters over the log.

*Definition 4.1 (Start and end counters for POT logs).* Let  $L$  be a POT log over  $\mathcal{A}$ . The *start counter*  $\text{start}_L : \mathcal{A} \rightarrow \mathbb{N}$  and the *end counter*  $\text{end}_L : \mathcal{A} \rightarrow \mathbb{N}$  are defined as

$$\text{start}_L(a) = \sum_{\pi=(E, \prec, \lambda) \in L} L(\pi) \cdot |\{e \in \min(\pi) \mid \lambda(e) = a\}|,$$

and

$$\text{end}_L(a) = \sum_{\pi=(E, \prec, \lambda) \in L} L(\pi) \cdot |\{e \in \max(\pi) \mid \lambda(e) = a\}|.$$

**4.1.2 Directly-follows evidence.** The directly-follows graph is a central abstraction in Inductive Miner cut detection. For partially ordered traces, we derive this evidence directly from the order relation. Cover relations contribute directly-follows evidence in their forced direction, since they represent immediate precedence constraints in the partial order. Incomparable event pairs contribute evidence in both directions, since either order is compatible with the trace.

*Definition 4.2 (Directly-follows graph for POT logs).* Let  $L$  be a POT log over  $\mathcal{A}$ . The *directly-follows graph*  $\text{DFG}_L : \mathcal{A} \times \mathcal{A} \rightarrow \mathbb{N}$  is defined as

$$\text{DFG}_L(a, b) = \sum_{\pi \in L} L(\pi) \cdot (\text{cov}_\pi(a, b) + \text{inc}_\pi(a, b)),$$

where, for  $\pi = (E, \prec, \lambda)$ ,

$$\text{cov}_\pi(a, b) = |\{(e, f) \in E \times E \mid e \prec^- f, \lambda(e) = a, \lambda(f) = b\}|$$

and

$$\text{inc}_\pi(a, b) = |\{\{e, f\} \subseteq E \mid e \parallel f, \{\lambda(e), \lambda(f)\} = \{a, b\}\}|.$$

The rule for incomparable events is the crucial bridge to existing cut detection. In sequence logs, concurrent activities often manifest as both  $a \rightarrow b$  and  $b \rightarrow a$  across different traces. In a POT log, this evidence is already present within a single trace: the two events are incomparable. Adding both arcs lets existing concurrency cut detectors observe the same bidirectionality without sampling or enumerating permutations.

**4.1.3 Eventually-follows evidence.** Some IM variants, especially partial-order cuts, use eventually-follows information to determine whether one block must precede another, whether two blocks are concurrent, or whether they should be merged. We therefore also lift eventually-follows evidence to partial orders.

*Definition 4.3 (Eventually-follows graph for POT logs).* Let  $L$  be a POT log over  $\mathcal{A}$ . The *eventually-follows graph*  $\text{EFG}_L : \mathcal{A} \times \mathcal{A} \rightarrow \mathbb{N}$  is defined as

$$\text{EFG}_L(a, b) = \sum_{\pi \in L} L(\pi) \cdot (\text{ord}'_\pi(a, b) + \text{inc}'_\pi(a, b)),$$

where, for  $\pi = (E, \prec, \lambda)$ ,

$$\text{ord}'_\pi(a, b) = |\{(e, f) \in E \times E \mid e \prec f, \lambda(e) = a, \lambda(f) = b\}|$$

and

$$\text{inc}'_\pi(a, b) = |\{\{e, f\} \subseteq E \mid e \parallel f, \{\lambda(e), \lambda(f)\} = \{a, b\}\}|.$$

Comparable event pairs contribute eventually-follows evidence in their forced direction. Incomparable event pairs contribute evidence in both possible directions.

**4.1.4 Discussion: Frequency Semantics.** Frequency handling for incomparable events is a design choice, and several reasonable normalizations are possible. For example, one could split the weight of an incomparable pair across the two directions, average over all linearizations, or use another weighting scheme tailored to a specific noise-filtering strategy. In our approach, we use a case-based maximal support semantics: a trace with multiplicity  $m$  contributes weight  $m$  to each forced relation, and if two events are incomparable, it contributes weight  $m$  to both possible directions. This choice reflects the fact that, if the partial order were replaced by one arbitrary compatible total order, either orientation of an incomparable pair could be selected by an admissible tie-breaking rule; hence, under a normalized variant that preserves the original case multiplicity rather than expanding the trace into all linearizations, either direction could receive the full support  $m$ . Our abstraction records this maximal normalized support for each direction, while avoiding the factorial bias of all-linearization counting. Other choices can certainly be justified, and they are orthogonal to our main contribution: any frequency normalization can be applied without changing the overall architecture.

## 4.2 Projection of Partially Ordered Traces

Recursive discovery requires not only detecting a cut, but also constructing the sublogs on which the recursive calls are applied. In the classical Inductive Miner, projection is operator-dependent: different cut types use different projection rules because they make different assumptions about how activities occur in a trace. We follow the same principle for partially ordered traces. We define two projection schemes, corresponding to the two POWL cut types used by the miner. We use *block projection* for partial-order constructs and *visit-projection* for choice-graphs. Basic process-tree cuts can be handled through these POWL constructs: sequence and concurrency cuts can be represented as partial orders, whereas exclusive-choice and loop cuts can be represented by choice graphs.

*Definition 4.4 (Block projection).* Let  $\pi = (E, \prec, \lambda)$  be a partially ordered trace and let  $A \subseteq \mathcal{A}$ . The *block projection* of  $\pi$  on  $A$  is the induced subtrace

$$\pi \upharpoonright_A^{\text{blk}} = (E_A, \prec_A, \lambda_A),$$

where

$$E_A = \{e \in E \mid \lambda(e) \in A\}, \quad \prec_A = \prec \cap (E_A \times E_A), \quad \lambda_A = \lambda|_{E_A}.$$

Block projection keeps all events of the selected activity group in one projected trace. It is used for partial-order cuts, where each cut block is interpreted as one component of the same execution. The projection therefore preserves the ordering constraints among all events belonging to that component.

For choice-graph cuts, block projection is not sufficient. A choice graph may contain cycles, and therefore the same activity group may be visited several times within one trace. If all events with labels in the group were collected into one induced sub-order, distinct visits could be merged into a single recursive instance. The projection used for choice graphs must therefore first identify which selected events belong to the same visit.

We identify such visits by a graph construction over the selected events. The graph connects selected events that should remain in the same projected trace: events that are locally adjacent in the partial order, and events that are incomparable. The former preserves local ordering structure, whereas the latter keeps concurrent selected events together. Selected events that are only related through events outside the group are not connected directly.

*Definition 4.5 (Projection graph).* Let  $\pi = (E, \prec, \lambda)$  be a partially ordered trace and let  $A \subseteq \mathcal{A}$ . The *projection graph* of  $\pi$  on  $A$  is the undirected graph

$$\Gamma_A(\pi) = (E_A, R_A),$$

where

$$E_A = \{e \in E \mid \lambda(e) \in A\}$$

and

$$\{e, f\} \in R_A \quad \text{iff} \quad e \neq f \text{ and } (e \prec^- f \vee f \prec^- e \vee e \parallel f).$$

The connected components of  $\Gamma_A(\pi)$  are denoted by  $\text{Comp}_A(\pi)$ .

Each connected component of the projection graph yields one projected partially ordered trace.

*Definition 4.6 (Visit projection).* Let  $\pi = (E, \prec, \lambda)$  be a partially ordered trace and let  $A \subseteq \mathcal{A}$ . The *visit projection* of  $\pi$  on  $A$  is the multiset

$$\pi \upharpoonright_A^{\text{vis}} = \left[ (C, \prec \cap (C \times C), \lambda|_C) \mid C \in \text{Comp}_A(\pi) \right].$$

Visit projection is used mainly for choice-graph cuts. Our implementation also uses the same component-based idea for  $\tau$ -loop fall-throughs. These projections differ only in how the projection graph is constructed before taking connected components. In the ordinary  $\tau$ -loop fall-through, connections through start activities are suppressed. In the strict  $\tau$ -loop fall-through, connections from end activities back to start activities are suppressed. Thus, the fall-through cases reuse the same graph-and-components projection principle, but with edge relations adapted to the corresponding fall-through semantics.

### 4.3 Algorithmic Summary

[Algorithms 1](#) and [2](#) summarize the complete pipeline. The preprocessing step constructs a POT log, while the discovery step recursively applies the abstraction and projection machinery introduced in [Sections 4.1](#) and [4.2](#). The construction of the POT log is independent of the miner: users may preprocess event data in any suitable way before discovery. For example, timestamps may be abstracted to the day level, domain-specific ordering constraints may be added, or accidental ordering evidence may be deliberately removed. If lifecycle information is available, e.g., events with start and completion states, then interval-based techniques can be used to derive the partial order. In such settings, different matching strategies can be applied to pair start and completion events. These extraction techniques are orthogonal to the discovery algorithm and can be integrated into the same pipeline as long as they produce partially ordered traces.

In [Algorithm 1](#), we use a deliberately simple timestamp-based extraction. For each unique case identifier  $c$ , every row  $(c, a, t)$  gives rise to a fresh event  $e$  with label  $\lambda_c(e) = a$  and timestamp  $\tau_c(e) = t$ . The partial order is then derived purely from timestamps: an event  $e$  precedes an event  $f$  iff  $\tau_c(e) + \Delta < \tau_c(f)$ , where  $\Delta$  is an optional tolerance window. Hence, events with equal timestamps are incomparable when  $\Delta = 0$ , and events whose timestamps are within the tolerance window are also treated as incomparable.

After extraction, discovery proceeds recursively. [Algorithm 2](#) computes the POT-aware start, end, directly-follows, and eventually-follows abstractions defined in [Section 4.1](#). The discovery variant  $v$  specifies an ordered list of decomposition functions. These functions include the standard cut detectors as well as fall-through functions. Each function is applied to the POT abstractions and may return a cut over an activity partitioning. The cut is then translated into the corresponding POWL construct: sequence and concurrency cuts are represented as partial-order constructs, whereas exclusive-choice and loop cuts are represented as choice-graph constructs. This translation determines the projection



**Algorithm 1:** Timestamp-based extraction of a partially ordered trace log.**Input:** Event table  $T$  with case column, activity column, timestamp column, and tolerance window  $\Delta \geq 0$ **Output:** POT log  $L$ 


---

```

1  $L \leftarrow []$ 
2 foreach unique case identifier  $c$  in the case column of  $T$  do
3    $E_c \leftarrow \emptyset, \prec_c \leftarrow \emptyset$ 
4   foreach row  $(c, a, t)$  of case  $c$  do
5     create a fresh event  $e$ 
6     add  $e$  to  $E_c$ 
7     set  $\lambda_c(e) = a$ 
8     set  $\tau_c(e) = t$ 
9   foreach pair of distinct events  $e, f \in E_c$  do
10    if  $\tau_c(e) + \Delta < \tau_c(f)$  then
11      add  $(e, f)$  to  $\prec_c$ 
12    $\pi_c \leftarrow (E_c, \prec_c, \lambda_c)$ 
13   add  $\pi_c$  to  $L$ 
14 return  $L$ 

```

---

**Algorithm 2:** POT inductive discovery.**Input:** POT log  $L$ ; discovery variant  $v$ **Output:** POWL model  $M = \text{POT-IM}(L, v)$ 


---

```

1 if a base case applies then
2   return baseModel( $L$ )
3  $\alpha \leftarrow (\text{start}_L, \text{end}_L, \text{DFG}_L, \text{EFG}_L)$ 
4 foreach cut detection mechanism  $f$  configured by  $v$  do
5   if  $f(\alpha)$  detects a cut  $(c; A_1, \dots, A_n)$  then
6      $p \leftarrow$  translate  $c$  into an equivalent POWL construct
7     if  $p$  is a partial-order then
8        $(L_1, \dots, L_n) \leftarrow \text{blockProject}(L; A_1, \dots, A_n)$ 
9     else
10       $(L_1, \dots, L_n) \leftarrow \text{visitProject}(L; A_1, \dots, A_n)$ 
11     foreach  $i \in \{1, \dots, n\}$  do
12        $M_i \leftarrow \text{POT-IM}(L_i, v)$ 
13     return compose( $p; M_1, \dots, M_n$ )

```

---

scheme used in the recursive step. Partial-order constructs use block projection, while choice-graph constructs use visit projection. The procedure is guaranteed to terminate because the configured list of decomposition functions includes a fall-through function that returns a model whenever no ordinary cut is detected. The resulting POT sublogs are then mined recursively.

## 5 Experimental Evaluation

The evaluation focuses on three central aspects of the proposed lifting of inductive discovery to partially ordered traces. First, we evaluate scalability against linearization: our approach should avoid the factorial blow-up caused by

materializing all total orders represented by a partially ordered trace. Second, we evaluate quality and representation bias on a real-life event log: sequence-based discovery must commit to an arbitrary tie-breaking order when events share the same timestamp, whereas our approach can preserve such ties as incomparabilities. Third, we evaluate the informational value of the partial-order representation itself in a controlled setting, by investigating whether preserving concurrency explicitly allows the intended process behavior to be captured from fewer observed executions than with sequential traces.

We use the default POWL inductive discovery configuration provided by the POWL package.

## 5.1 Scalability on Synthetic Partially Ordered Traces

**5.1.1 Setup.** The scalability experiment uses the simple partially ordered pattern from Figure 1. Each case consists of an initial activity  $A$ , followed by a concurrent block of width  $k$ , followed by a final activity  $C$ :

$$A ; (B_1 \parallel \dots \parallel B_k) ; C.$$

For every value of  $k$ , the POT log contains one case with one partially ordered trace. This single trace has  $k + 2$  events and compactly represents  $k!$  sequential executions. We consider  $k \in \{2, 4, 6, 8, 10\}$ , ranging from two compatible sequential traces for  $k = 2$  to  $10! = 3,628,800$  for  $k = 10$ .

We compare three settings:

- (1) **POT-IM:** the proposed method, which consumes the partially ordered trace directly.
- (2) **single linearization:** the sequence-based miner that treats the order of events in the input trace as the actual sequential order.
- (3) **all linearizations:** the sequence-based baseline preceded by a preprocessing step that materializes all topological orderings of the partially ordered trace.

For the all-linearizations baseline, we report the time needed to materialize the linearized log separately from the subsequent IM discovery time. For all three settings, we also report whether the concurrent block  $B_1, \dots, B_k$  is detected in the discovered model.

**5.1.2 Results.** Table 1 shows the scalability results. For the POT-based approach and the single-linearization baseline, discovery remains fast across all considered values of  $k$ . The all-linearizations baseline, in contrast, becomes increasingly expensive as the degree of concurrency grows. At  $k = 10$ , materializing all linearizations takes about 32.2 minutes, and the subsequent IM discovery step takes another 10 minutes.

Overall, the experiment illustrates how enumerating all linearizations exposes the same concurrency evidence to the sequence-based miner, but only at factorial cost. Using only one raw-order sequence is efficient, but introduces a sequential dependency that is not present in the partially ordered trace. POT-IM combines the advantages of both alternatives: it preserves the concurrency information of the partially ordered trace while avoiding the factorial overhead of materializing all compatible linearizations.

## 5.2 Quality and Tie-Breaking Bias on BPI Challenge 2012

**5.2.1 Setup.** The second experiment evaluates the effect of arbitrary timestamp tie-breaking on a real-life event log. We use the BPI Challenge 2012 event log [5], because it contains events that share identical timestamps. This experiment is not intended to evaluate lifecycle-based partial-order extraction. Although lifecycle information can in principle be used to derive interval-based concurrency, most activities in this log are available only with completion timestamps. We

Table 1. Scalability on the synthetic concurrency benchmark.

| $k$ | Variant              | Input log |            | Runtime (s)   |              | Concurrency detected? |
|-----|----------------------|-----------|------------|---------------|--------------|-----------------------|
|     |                      | cases     | events     | linearization | IM discovery |                       |
| 2   | POT-IM               | 1         | 4          | –             | 0.004        | yes                   |
|     | single linearization | 1         | 4          | –             | 0.007        | no                    |
|     | all linearizations   | 2         | 8          | 0.006         | 0.004        | yes                   |
| 4   | POT-IM               | 1         | 6          | –             | 0.003        | yes                   |
|     | single linearization | 1         | 6          | –             | 0.004        | no                    |
|     | all linearizations   | 24        | 144        | 0.006         | 0.008        | yes                   |
| 6   | POT-IM               | 1         | 8          | –             | 0.008        | yes                   |
|     | single linearization | 1         | 8          | –             | 0.006        | no                    |
|     | all linearizations   | 720       | 5,760      | 0.164         | 0.060        | yes                   |
| 8   | POT-IM               | 1         | 10         | –             | 0.004        | yes                   |
|     | single linearization | 1         | 10         | –             | 0.005        | no                    |
|     | all linearizations   | 40,320    | 403,200    | 3.802         | 2.351        | yes                   |
| 10  | POT-IM               | 1         | 12         | –             | 0.008        | yes                   |
|     | single linearization | 1         | 12         | –             | 0.007        | no                    |
|     | all linearizations   | 3,628,800 | 43,545,600 | 1,929.244     | 600.539      | yes                   |

Table 2. Alignment-based conformance quality on BPI Challenge 2012.  $\Delta$  denotes the absolute difference between the scores on  $L_{asc}$  and  $L_{des}$ . Lowest  $\Delta$  values per filtering setting and metric are shown in bold.

| Filtering | Discovered model    | Average trace fitness |           |              | Fitting traces (%) |           |          | Precision |           |              |
|-----------|---------------------|-----------------------|-----------|--------------|--------------------|-----------|----------|-----------|-----------|--------------|
|           |                     | $L_{asc}$             | $L_{des}$ | $\Delta$     | $L_{asc}$          | $L_{des}$ | $\Delta$ | $L_{asc}$ | $L_{des}$ | $\Delta$     |
| –         | Seq-IM( $L_{asc}$ ) | 1                     | 0.991     | 0.009        | 100                | 80.6      | 19.4     | 0.220     | 0.253     | 0.033        |
|           | Seq-IM( $L_{des}$ ) | 0.971                 | 1         | 0.029        | 64.8               | 100       | 35.2     | 0.252     | 0.219     | 0.033        |
|           | POT-IM              | 1                     | 1         | <b>0</b>     | 100                | 100       | <b>0</b> | 0.174     | 0.178     | <b>0.004</b> |
| 0.8       | Seq-IM( $L_{asc}$ ) | 0.852                 | 0.834     | 0.018        | 26.2               | 26.2      | <b>0</b> | 0.712     | 0.692     | 0.020        |
|           | Seq-IM( $L_{des}$ ) | 0.812                 | 0.842     | 0.030        | 26.2               | 26.2      | <b>0</b> | 0.696     | 0.651     | 0.046        |
|           | POT-IM              | 0.834                 | 0.827     | <b>0.007</b> | 26.2               | 26.2      | <b>0</b> | 0.669     | 0.674     | <b>0.005</b> |

therefore keep completion events only and construct a POT log using the timestamp-based extraction of [Algorithm 1](#) with tolerance window  $\Delta = 0$ . Events of the same case with identical timestamps are thus treated as incomparable.

We compare this POT representation with two sequential versions of the same log. Both sequential logs order events by timestamp. Ties are resolved either by ascending activity label, yielding  $L_{asc}$ , or by descending activity label, yielding  $L_{des}$ . Hence, the two sequential logs contain the same cases, events, activities, and timestamps; they differ only in the arbitrary order assigned to timestamp ties.

For each input representation, we discover one model without filtering and one model with filtering threshold 0.8. Each discovered model is evaluated on both  $L_{asc}$  and  $L_{des}$  using alignment-based conformance checking. We use PM4Py [\[4\]](#) to compute average trace fitness, the percentage of fitting traces, and precision.

**5.2.2 Results.** [Table 2](#) shows that sequence-based discovery is sensitive to the chosen tie-breaking convention. Without filtering, the model discovered from  $L_{asc}$  fits  $L_{asc}$  perfectly, but its fitting-trace score drops to 80.6% on  $L_{des}$ . Conversely, the model discovered from  $L_{des}$  fits  $L_{des}$  perfectly, but drops to 64.8% on  $L_{asc}$ . Thus, the sequential miner may learn

ordering artifacts introduced only by the tie-breaking rule. The POT-based model avoids this dependency. Since timestamp ties are kept incomparable during discovery, the same model obtains perfect average trace fitness and fitting-trace scores on both sequential realizations. Its precision is lower than that of the sequence-based models, which is expected: more permissive concurrency generally reduces precision. However, the POT-based model is also the most stable with respect to precision, with the smallest precision difference across the two evaluation logs.

With filtering threshold 0.8, the absolute scores become more balanced. All models obtain the same percentage of fitting traces on both sequential logs, and the model discovered from  $L_{asc}$  achieves the highest absolute fitness and precision values. Nevertheless, the POT-based model remains the least sensitive to the tie-breaking convention.

Overall, the experiment confirms that treating timestamp ties as incomparabilities reduces the dependence of discovery results on arbitrary tie-breaking. POT-IM preserves the ordering information supported by the data while avoiding sequential dependencies introduced only by a chosen linearization. This comes with an expected loss of precision, because the model allows behavior corresponding to multiple compatible orderings of tied events. However, this additional behavior reflects uncertainty already present in the event data rather than behavior introduced by the discovery algorithm itself.

### 5.3 Representation Efficiency under Paired Simulation

**5.3.1 Motivation.** The real-life tie-breaking experiment in Section 5.2 demonstrates why preserving incomparability can be useful when the event data themselves do not determine a unique order. However, such an experiment alone does not isolate whether the advantage comes from the partial-order representation itself or merely from avoiding a particular data-quality artifact. We therefore add a controlled experiment in which both input representations are generated from exactly the same process executions. The only difference is whether concurrency is retained explicitly or linearized.

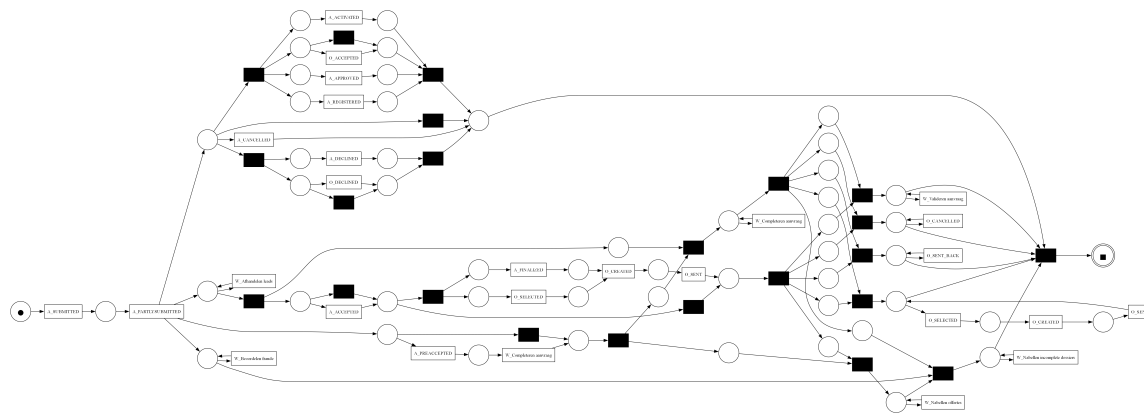
The intuition is straightforward. If two activities are concurrent, one POT can encode this directly through incomparability. A sequence cannot: it must show either one activity before the other or vice versa. A sequence-based miner therefore needs multiple executions with sufficiently diverse interleavings before bidirectional ordering evidence becomes visible. The experiment tests whether this difference translates into faster coverage of the behavior generated by the reference model.

**5.3.2 Reference Model and Paired Simulation.** We use the BPI Challenge 2012 event log [5] to obtain a concurrency-rich model with the partial-order aggregation-based discovery approach [10]. We then *freeze* the resulting POWL model and use it solely as an assumed reference process for controlled simulation. The corresponding Petri net is shown in Figure 2. Importantly, the experiment does not claim that this model is the true process underlying BPI Challenge 2012; its role is to provide a realistic and concurrency-rich process whose behavior is fully controlled during simulation.

From this fixed model, we simulate 1,000 non-empty executions. Each sampled execution is first retained as a partially ordered trace  $\pi_i$  and is then randomly topologically sorted to obtain one compatible sequential trace  $\sigma_i$ . Consequently,

$$\sigma_i \in \text{Lin}(\pi_i) \quad \text{for every } i \in \{1, \dots, 1000\}.$$

The POT and sequential logs therefore contain the same sampled branch choices, optional behavior, repetitions, and executed activities. Their only systematic difference is the representation of concurrent events: the POT keeps them incomparable, whereas the sequential trace commits to one random compatible order. We use a fixed random seed for reproducibility.



We then increase the amount of discovery input gradually. For each  $n \in \mathcal{N} = \{1, 2, \dots, 30, 40, 50, 60, 70, 80, 90, 100\}$ , we discover one model from the first  $n$  partial-order traces and one model from the corresponding  $n$  sequential traces. Thus, at every input size, both discovery runs are based on equally many executions of the same reference process.

**5.3.3 Results.** Figure 3 shows a pronounced sample-efficiency advantage for the POT representation. The percentage of perfectly fitting traces rises rapidly for POT-based discovery: with  $n = 9$  input POTs, all 1,000 sequential reference traces already fit perfectly, and this 100% coverage is retained for every larger input size considered. Sequential discovery requires substantially more observations. It first reaches 100% perfectly fitting traces only around  $n = 24$  and is not yet stable: the score drops again for the inputs around  $n = 27$ –28 before returning to 100% from  $n = 29$  onward.

The two panels capture complementary aspects of the same effect. Perfectly fitting traces provide the strongest evidence that the POT representation allows the miner to cover the behavior generated by the reference process from few executions. Behavioral similarity then adds a model-level perspective: even once both approaches can replay all sampled reference traces, the models discovered from POTs remain behaviorally closer to the process from which the executions were generated.

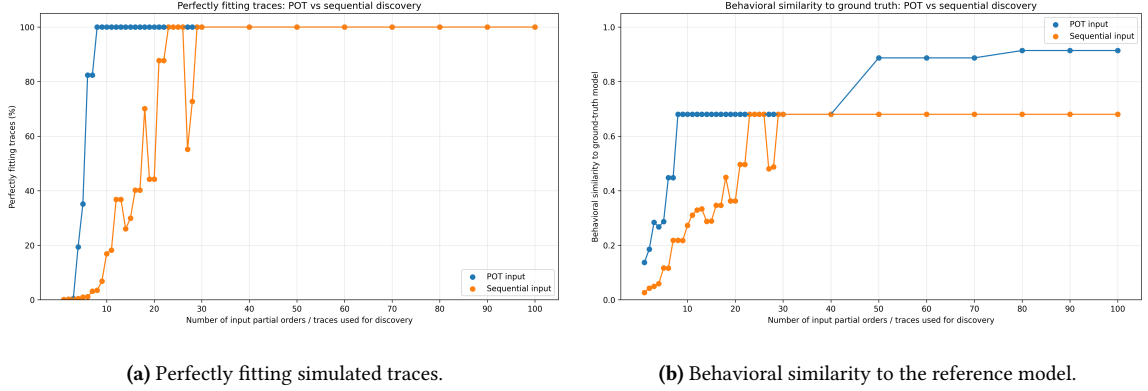


Fig. 3. Controlled paired-simulation experiment. For every  $n$ , the POT and sequential discovery inputs describe the same  $n$  simulated executions; only the representation differs. **(a)** Percentage of the common 1,000-trace sequential simulation log that fits the discovered model perfectly. **(b)** Behavioral similarity between the discovered model and the fixed reference model. The “ground truth” wording inside panel (b) refers to this fixed assumed reference model.

Crucially, this experiment isolates a mechanism that is different from correcting noisy or ambiguous event data. The sequential traces are not incorrect: each is a valid linearization of exactly the same underlying execution represented by its paired POT. Nevertheless, the sequence-based miner must reconstruct this concurrency indirectly from variation across several traces, whereas the POT-based miner can observe it directly within one execution. The earlier replay coverage and higher model-level similarity therefore provide evidence that partial orders can be more information-efficient observations for process discovery, not only a remedy for arbitrary timestamp tie-breaking.

## 6 Conclusion

This paper presented a lifting of inductive process discovery from totally ordered traces to partially ordered traces. The proposed approach enables Inductive Mining discovery to operate directly on partial-order event data, avoiding the need to select arbitrary linearizations or enumerate all compatible total orders. To achieve this, we redefined the abstraction layer used by inductive discovery, including start and end activities, directly-follows evidence, and eventually-follows evidence, so that both forced precedence constraints and incomparabilities are reflected in the mined behavior. We also introduced projection mechanisms for partially ordered traces, allowing recursive discovery to preserve partial-order information throughout the mining procedure. The resulting POT-aware inductive miner remains compatible with the recursive decomposition principle of existing inductive discovery techniques while addressing the representational bias and combinatorial overhead caused by linearization.

Future work can extend the proposed framework in several directions. First, alternative frequency semantics for incomparable event pairs should be investigated, including normalized, probabilistic, or noise-aware weighting schemes. Second, the current projection mechanisms can be generalized to additional process constructs, enabling a broader range of inductive discovery variants to consume partially ordered traces directly. Finally, more advanced extraction techniques should be integrated, especially lifecycle- and interval-based methods that derive partial orders from overlapping activity executions.

## Acknowledgments

This work was supported by the Federal Ministry of Research, Technology and Space (BMFTR), Germany (grant 16IS23065).

## References

- [1] Augusto, A., Dumas, M., Rosa, M.L.: Automated discovery of process models with true concurrency and inclusive choices. In: ICPM 2020 Workshops. pp. 43–56 (2020). [https://doi.org/10.1007/978-3-030-72693-5\\_4](https://doi.org/10.1007/978-3-030-72693-5_4)
- [2] Bergenthum, R.: Prime miner - process discovery using prime event structures. In: ICPM 2019. pp. 41–48 (2019). <https://doi.org/10.1109/ICPM.2019.00017>
- [3] Bergenthum, R., Mauser, S.: Folding partially ordered runs. In: Applications of Region Theory 2011. pp. 52–62 (2011)
- [4] Berti, A., van Zelst, S.J., Schuster, D.: PM4Py: A process mining library for python. *Softw. Impacts* **17**, 100556 (2023). <https://doi.org/10.1016/J.SIMPA.2023.100556>
- [5] van Dongen, B.: BPI Challenge 2012 (2012). <https://doi.org/10.4121/UUID:3926DB30-F712-4394-AEBC-75976070E91F>
- [6] van Dongen, B.F., Desel, J., van der Aalst, W.M.P.: Aggregating causal runs into workflow nets. *Trans. Petri Nets Other Model. Concurr.* **6**, 334–363 (2012). [https://doi.org/10.1007/978-3-642-35179-2\\_14](https://doi.org/10.1007/978-3-642-35179-2_14)
- [7] Dumas, M., Garcia-Bañuelos, L.: Process mining reloaded: Event structures as a unified representation of process models and event logs. In: PETRI NETS 2015. pp. 33–48 (2015). [https://doi.org/10.1007/978-3-319-19488-2\\_2](https://doi.org/10.1007/978-3-319-19488-2_2)
- [8] Folz-Weinstein, S., Bergenthum, R., Desel, J., Kovár, J.: ILP<sup>2</sup> miner - process discovery for partially ordered event logs using integer linear programming. In: PETRI NETS 2023. pp. 59–76 (2023). [https://doi.org/10.1007/978-3-031-33620-1\\_4](https://doi.org/10.1007/978-3-031-33620-1_4)
- [9] Folz-Weinstein, S., Rennert, C., Mannel, L.L., Bergenthum, R., van der Aalst, W.M.P.: est<sup>2</sup> miner - process discovery based on firing partial orders. In: Advanced Information Systems Engineering - 37th International Conference, CAISE 2025, Vienna, Austria, June 16–20, 2025, Proceedings, Part II. pp. 59–75 (2025). [https://doi.org/10.1007/978-3-031-94571-7\\_4](https://doi.org/10.1007/978-3-031-94571-7_4)
- [10] Kourani, H., Park, G., van der Aalst, W.M.P.: Revealing inherent concurrency in event data: A partial order approach to process discovery. *CoRR abs/2509.15346* (2025). <https://doi.org/10.48550/ARXIV.2509.15346>
- [11] Kourani, H., Park, G., van der Aalst, W.M.P.: Unlocking non-block-structured decisions: Inductive mining with choice graphs. In: Business Process Management - 23rd International Conference, BPM 2025, Seville, Spain, August 31 - September 5, 2025, Proceedings. pp. 144–161 (2025). [https://doi.org/10.1007/978-3-032-02867-9\\_10](https://doi.org/10.1007/978-3-032-02867-9_10)
- [12] Kourani, H., Park, G., van der Aalst, W.M.P.: A discovery technique for expressive yet sound process models. *Process Sci.* **3**(1), 14 (2026). <https://doi.org/10.1007/S44311-026-00046-8>
- [13] Kourani, H., Schuster, D., van der Aalst, W.M.P.: Scalable discovery of partially ordered workflow models with formal guarantees. In: ICPM 2023. pp. 89–96 (2023). <https://doi.org/10.1109/ICPM60904.2023.10271941>
- [14] Kourani, H., van Zelst, S.J.: POWL: partially ordered workflow language. In: BPM 2023. pp. 92–108 (2023). [https://doi.org/10.1007/978-3-031-41620-0\\_6](https://doi.org/10.1007/978-3-031-41620-0_6)
- [15] Kourani, H., van Zelst, S.J., Schuster, D., van der Aalst, W.M.P.: Discovering partially ordered workflow models. *Inf. Syst.* **128**, 102493 (2025). <https://doi.org/10.1016/J.IS.2024.102493>
- [16] Kovár, J., Bergenthum, R.: Exploratory process discovery for petri nets. In: AWPEN 2024 (2024)
- [17] Leemans, S.J.J.: Robust Process Mining with Guarantees - Process Discovery, Conformance Checking and Enhancement, Lecture Notes in Business Information Processing, vol. 440 (2022). <https://doi.org/10.1007/978-3-030-96655-3>
- [18] Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Using life cycle information in process discovery. In: BPM 2015 Workshops. pp. 204–217 (2015). [https://doi.org/10.1007/978-3-319-42887-1\\_17](https://doi.org/10.1007/978-3-319-42887-1_17)
- [19] Leemans, S.J.J., van Zelst, S.J., Lu, X.: Partial-order-based process mining: a survey and outlook. *Knowl. Inf. Syst.* **65**(1), 1–29 (2023). <https://doi.org/10.1007/S10115-022-01777-3>
- [20] de León, H.P., Rodríguez, C., Carmona, J., Heljanko, K., Haar, S.: Unfolding-based process discovery. In: ATVA 2015. pp. 31–47 (2015). [https://doi.org/10.1007/978-3-319-24953-7\\_4](https://doi.org/10.1007/978-3-319-24953-7_4)
- [21] Rennert, C., Pourbafrani, M., van der Aalst, W.M.P.: Evaluation of study plans using partial orders. In: Process Mining Workshops - ICPM 2024 International Workshops, Lyngby, Denmark, October 14–18, 2024, Revised Selected Papers. pp. 154–166 (2024). [https://doi.org/10.1007/978-3-031-82225-4\\_12](https://doi.org/10.1007/978-3-031-82225-4_12)
- [22] Schuster, D., Schade, L., van Zelst, S.J., van der Aalst, W.M.P.: Visualizing trace variants from partially ordered event data. In: Process Mining Workshops - ICPM 2021 International Workshops, Eindhoven, The Netherlands, October 31 - November 4, 2021, Revised Selected Papers. pp. 34–46 (2021). [https://doi.org/10.1007/978-3-030-98581-3\\_3](https://doi.org/10.1007/978-3-030-98581-3_3)
- [23] Schuster, D., van Zelst, S.J., van der Aalst, W.M.P.: Cortado: A dedicated process mining tool for interactive process discovery. *SoftwareX* **22**, 101373 (2023). <https://doi.org/10.1016/J.SOFTX.2023.101373>
- [24] Schuster, D., Zerbato, F., van Zelst, S.J., van der Aalst, W.M.P.: Defining and visualizing process execution variants from partially ordered event data. *Inf. Sci.* **657**, 119958 (2024). <https://doi.org/10.1016/J.INS.2023.119958>

- [25] Van Dongen, B.F., Van der Aalst, W.M.: Multi-phase process mining: Aggregating instance graphs into EPCs and Petri nets. In: PNCWB 2005. pp. 35–58 (2005)
- [26] Wil M. P. van der Aalst et al.: Process mining manifesto. In: Business Process Management Workshops - BPM 2011 International Workshops, Clermont-Ferrand, France, August 29, 2011, Revised Selected Papers, Part I. pp. 169–194 (2011). [https://doi.org/10.1007/978-3-642-28108-2\\_19](https://doi.org/10.1007/978-3-642-28108-2_19)